Лабораторная работа №4. Frontend на JavaScript-фреймворке

Цель

В рамках лабораторной работы необходимо перенести frontend системы управления студентами из лабораторной работы №1 на современный JavaScript-фреймворк, сохранив основную функциональность и добавив авторизацию, разграничение прав доступа, управление состоянием и реактивное программирование.

Приложение должно взаимодействовать с REST API, разработанным в лабораторной работе №3.

Варианты реализации

В зависимости от варианта необходимо использовать один из следующих фреймворков:

- React;
- Vue;
- Angular;
- Svelte.

Использование библиотеки RxJS обязательно для всех вариантов.

Работа со студентами

Необходимо сохранить функциональность лабораторной работы №1.

Приложение должно поддерживать просмотр списка студентов и подробной информации об отдельном студенте, создание, редактирование и удаление записей.

При работе со списком необходимо реализовать поиск, фильтрацию по всем полям, сортировку и пагинацию.

Все данные должны загружаться через REST API лабораторной работы №3. Изменения должны отображаться без перезагрузки страницы.

Авторизация и JWT

Необходимо реализовать регистрацию, авторизацию, выход из системы и просмотр информации о текущем пользователе.

Обязательные страницы:

- `/login` — авторизация;
- `/register` — регистрация;
- `/profile` — профиль пользователя.

Для авторизации необходимо использовать JWT с поддержкой access- и refresh-токенов.

Компонентная архитектура

Приложение должно быть построено с использованием компонентов выбранного фреймворка.

Необходимо использовать локальное состояние компонентов, передачу данных через props / inputs, hooks или lifecycle-механизмы, условный рендеринг и клиентскую маршрутизацию.

Конкретная структура компонентов и организация проекта определяются разработчиком.

Управление состоянием

Необходимо использовать глобальное состояние приложения.

Допустимые варианты:

- React — Context API / Redux Toolkit / Zustand.
- Vue — Pinia.
- Angular — Signals / NgRx.
- Svelte — Stores / Runes.

Реактивное программирование

В приложении необходимо реализовать реактивную обработку пользовательских событий и асинхронных данных с использованием **RxJS**.

Для реализации необходимо использовать Observable, механизмы подписки и операторы RxJS, соответствующие решаемым задачам.

Реактивная логика должна быть отделена от логики отображения интерфейса и организована таким образом, чтобы ее можно было повторно использовать.

Простое подключение RxJS без практического применения в указанных сценариях не считается выполнением требования.

Валидация и обработка ошибок

Необходимо реализовать клиентскую валидацию форм и обработку ошибок backend.

Приложение должно корректно обрабатывать ошибки авторизации, недостаточность прав доступа, истечение срока действия токенов, ошибки валидации, отсутствие запрашиваемых данных, недоступность сервера и внутренние ошибки backend.

Пользователь должен получать понятные сообщения об ошибках.

При выполнении асинхронных операций должна отображаться индикация загрузки. Необходимо предусмотреть защиту от повторной отправки форм во время выполнения запросов.

Контейнеризация и запуск

Все приложение, включая frontend лабораторной работы №4 и backend лабораторной работы №3, должно запускаться совместно в Docker с помощью Docker Compose.

Необходимо:

- Создать `Dockerfile` для сборки и запуска приложения.
- Использовать многоэтапную сборку (multi-stage build).
- Настроить раздачу собранного frontend-приложения через Nginx или аналогичный веб-сервер.
- Создать `docker-compose.yml` для совместного запуска frontend и backend лабораторной работы №3.
- Обеспечить взаимодействие frontend с backend через REST API, включая корректную маршрутизацию запросов.
- Вынести параметры конфигурации, зависящие от окружения, за пределы исходного кода.

Приложение должно запускаться командой `docker compose up --build` без необходимости ручной настройки контейнеров.

Необходимо предоставить файл `.env.example` с примером конфигурации и `README.md` с инструкциями по сборке, запуску и остановке приложения.

Вопросы к защите

1. JavaScript-фреймворки: React, Vue, Angular, Svelte. Основные особенности и различия.
2. SPA и клиентская маршрутизация. Принципы работы одностраничных приложений.
3. Компонентная архитектура. Компоненты, props, события и взаимодействие между компонентами.
4. Жизненный цикл компонентов. Hooks и lifecycle-механизмы.
5. Рендеринг интерфейса. Virtual DOM, reconciliation и условный рендеринг.
6. Реактивность во frontend. Механизмы отслеживания изменений и обновления интерфейса.
7. Управление состоянием. Локальное и глобальное состояние, инструменты state management.
8. Реактивное программирование. Основные принципы и отличия от императивного подхода.
9. RxJS. Observable, Observer, Subject, BehaviorSubject и Subscription.
10. Операторы RxJS. Преобразование, фильтрация, комбинирование и обработка потоков данных.
11. Асинхронные потоки. Hot и cold Observable, управление подписками и предотвращение утечек памяти.
12. Реактивная обработка пользовательских событий. Поиск, фильтрация и отмена устаревших запросов.
13. Клиентская валидация, обработка ошибок и управление состояниями загрузки.
14. Аутентификация и авторизация. JWT, access- и refresh-токены.
15. Защита маршрутов и разграничение доступа на основе ролей.
16. Безопасность frontend-приложений. XSS, CSRF и основные способы защиты.
17. Архитектура frontend-приложения. Разделение ответственности и организация проекта.
18. Docker и контейнеризация. Dockerfile, образы и контейнеры.
19. Многоэтапная сборка Docker. Сборка frontend и раздача статических файлов через Nginx.
20. Docker Compose. Сети контейнеров, переменные окружения и взаимодействие frontend с backend.